Risk, generalisation and model selection

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

This note turns the informal goal "predict well on new data" into mathematics and into a working procedure. It defines expected and empirical risk, proves that a test-set estimate is unbiased and computes its standard error, explains why training error is optimistic, and shows how to estimate generalisation honestly with hold-out data, cross-validation, leave-one-out, sequential and nested validation. Capacity, under- and overfitting, the approximation–estimation and bias–variance decompositions and regularisation explain how to choose a model; grid and random search organise the choice. Every procedure comes with pseudocode and its cost, and every idea with a small example that can be redone by hand. The double-descent curve and scaling laws show how deep learning has refined, without overturning, this classical picture. This is the methodological core of the course: every later method is selected and evaluated in this way.

1 A learning algorithm has three ingredients

Almost every learning algorithm combines, explicitly or implicitly,

1. a **family of functions** $\mathcal{F} = \{f_\theta\}$ indexed by parameters θ;
2. a **loss** $L(f(\mathbf{x}), y)$ that measures the cost of predicting $f(\mathbf{x})$ when the truth is y (Note 1, Table 2);
3. an **optimisation procedure** that searches $\mathcal{F}$ for a function with small average loss on the training data.

The families met in this course grow from the very small to the very large. A histogram classifier (Note 3) predicts one value per cell, a staircase function. A k-nearest-neighbour (k-NN) predictor answers a query $\mathbf{x}$ with the average target (regression) or the majority label (classification) of the k training points nearest to $\mathbf{x}$; 1-NN is the case $k = 1$. It and the Parzen predictor (Note 5) are weighted sums of kernels centred on the training points, and so are the kernel machines of Note 10. A fit of a calibration curve uses polynomials, and a linear regression or classifier (Notes 8 and 9) uses affine functions $f(\mathbf{x}) = \mathbf{w}^{\mathsf{T}}\mathbf{x}+b$ — with the constant $f(\mathbf{x}) = b$ as the degenerate case; a decision tree (Note 3) is a histogram whose cells adapt to the data. A neural network is the extreme: its architecture fixes the family, its weights pick a member. Choosing the family is the most consequential decision in a project, and it is the subject of this note.

The three ingredients are independent choices. The same family can be trained with different losses, and the same loss can be minimised by different optimisers — in closed form, by enumeration or by gradient descent (Note 7). The smallest possible example already shows how much the loss matters.

Worked example — the best constant under three losses

Take the family of constants, $f(\mathbf{x}) = b$, and the training targets $y = 1, 2, 2, 3, 7$ ($n = 5$). The training objective is $J(b) = \frac{1}{n}\sum_i L(b, y_i)$.
Squared loss. $J(b) = \frac{1}{5}\sum_i (b - y_i)^2$ and

$$J'(b) = \frac{2}{n}\sum_i (b - y_i) = 2\,(b - \bar{y}) = 0 \quad \Longrightarrow \quad \hat{b} = \bar{y} = 3.$$

$J''(b) = 2 > 0$, so this is a minimum. The training risk is $\frac{1}{5}(4 + 1 + 1 + 0 + 16) = 4.4$.
Absolute loss. $J(b) = \frac{1}{5}\sum_i |b - y_i|$. Away from the data points the derivative of $|b - y_i|$ is $+1$ if $b > y_i$ and -1 if $b < y_i$, so $J'(b) = \frac{1}{n}\big[\#\{y_i < b\} - \#\{y_i > b\}\big]$. It changes sign where as many targets lie below b as above: $\hat{b}$ is the *median*, 2, with training risk $\frac{1}{5}(1+0+0+1+5) = 1.4$. The mean, $b = 3$, would give 1.6; the median would give squared risk 5.4 instead of 4.4.
0–1 loss (for class labels). $J(b)$ is the fraction of labels different from b, minimised by the most frequent label — the *majority class*.
Each loss picks a different "centre" of the data: the mean is pulled by the outlier 7, the median is not. Which is right depends on what the prediction is for, not on the data.

2 Measuring quality

What we ultimately care about is rarely the loss the algorithm minimises. Take a jet tagger in a collider analysis. What matters in the end is the precision of the measurement it serves — the uncertainty on a cross-section, or the expected significance of a search. One level down is a metric specific to the field, such as the background rejection at a fixed signal efficiency. Below that sit generic metrics (error rate, mean squared error), and at the bottom the loss the training actually minimises, usually a cross-entropy. Each level is a proxy for the one above it, and the gaps between them deserve attention.

Surrogate losses. The classification error $\mathbf{1}\{f(\mathbf{x}) \neq y\}$ is piecewise constant in the parameters, so its gradient is zero almost everywhere; and when the classes are not linearly separable, no efficient algorithm is known for finding the linear classifier with the fewest training errors (the problem is NP-hard). One therefore trains a real-valued score g and minimises a continuous, convex *surrogate* with two properties: it upper-bounds (or closely tracks) the 0–1 error, and its slope is non-zero wherever a decision is wrong or only barely right — exactly where the 0–1 loss is flat but the decision should change — so that a (sub)gradient points towards fewer errors. (The hinge $\max(0, 1 - t\,g)$ below has a kink at $t\,g = 1$ and is flat beyond it, where decisions are already confidently right.) With $t = 2y - 1 \in \{\pm 1\}$ for binary labels, the four common choices are:

Surrogate	Loss on one example	Output → decision	Used by
hinge	$\max(0,\ 1 - t\,g(\mathbf{x}))$	score g; $\mathbf{1}\{g > 0\}$	SVM (Note 9)
binary cross-entropy	$-y \log g - (1 - y)\log(1 - g)$	$g \approx P(Y = 1 \mid \mathbf{x})$; $\mathbf{1}\{g > \frac{1}{2}\}$	logistic regression (Note 9)
multiclass margin	$\max\big(0,\ 1 + \max_{k \neq y} g_k - g_y\big)$	scores $g_1, \ldots, g_m$; $\arg\max_k g_k$	multiclass SVM
cross-entropy	$-\log g_y(\mathbf{x})$	$g_k \approx P(Y = k \mid \mathbf{x})$; $\arg\max_k g_k$	softmax classifiers, networks

The cross-entropies are negative log-likelihoods of a categorical model (Note 2, Section 2). Probabilistic outputs have an extra benefit: the threshold can be moved after training to trade one kind of error for another.

Losses as functions of the margin. For a binary score write $z = t\,g(\mathbf{x})$, the *margin*: $z > 0$ means a correct decision, $z < 0$ a wrong one, and $|z|$ the confidence. Figure 1 plots the losses against z. If the score is a log-odds, $g = \log \frac{P(Y=1|\mathbf{x})}{P(Y=0|\mathbf{x})}$, then $P(Y = 1 \mid \mathbf{x}) = 1/(1+e^{-g})$ and $P(Y = 0 \mid \mathbf{x}) = 1 - P(Y = 1 \mid \mathbf{x}) = 1/(1 + e^{g})$. For $y = 1$ ($t = +1$) the cross-entropy is $-\log P(Y = 1 \mid \mathbf{x}) = \log(1 + e^{-g})$; for $y = 0$ ($t = -1$) it is $-\log P(Y = 0 \mid \mathbf{x}) = \log(1+e^{g})$. Both cases are $\log(1+e^{-tg}) = \log(1+e^{-z})$. Measured in bits (logarithm to base 2) it equals 1 at $z = 0$ and decreases, so it is *at least* 1 whenever the decision is wrong; the hinge $1 - z$ is also at least 1 there. Both surrogates are therefore upper bounds on the 0–1 loss, and driving them down drives the error down. Unlike the 0–1 loss they keep penalising correct but unconfident decisions ($0 < z < 1$), which is what gives them useful gradients. The squared loss $(1 - z)^2$ is an upper bound too, but it also penalises decisions that are "too right" ($z > 1$), a weakness discussed in Note 9, Section 2. Since $t^2 = 1$, $(t - g)^2 = t^2(1 - tg)^2 = (1 - z)^2$: this is the squared loss on the *score*, not the squared loss on a probability (the Brier score) used in the worked example below.

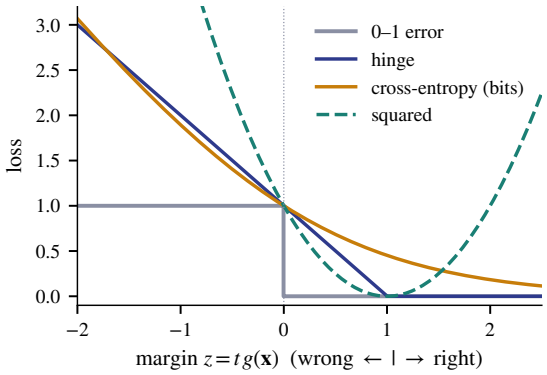


Figure 1. Binary classification losses as functions of the margin $z = t\,g(\mathbf{x})$. Hinge, cross-entropy in bits and squared loss all lie above the 0–1 loss for $z \le 0$.

Worked example — one prediction table, three losses

A classifier outputs $g_i \approx P(Y = 1 \mid \mathbf{x}_i)$ for five validation examples. The decision is $\mathbf{1}\{g_i > \frac{1}{2}\}$; the squared loss is applied to the probability, $(g_i - y_i)^2$ (this is called the *Brier score*); the cross-entropy uses the natural logarithm.

i	g_i	y_i	0–1 loss	squared loss $(g_i - y_i)^2$	cross-entropy
1	0.9	1	0	$0.1^2 = 0.01$	$-\ln 0.9 = 0.105$
2	0.6	1	0	$0.4^2 = 0.16$	$-\ln 0.6 = 0.511$
3	0.3	0	0	$0.3^2 = 0.09$	$-\ln 0.7 = 0.357$
4	0.2	1	1	$0.8^2 = 0.64$	$-\ln 0.2 = 1.609$
5	0.7	0	1	$0.7^2 = 0.49$	$-\ln 0.3 = 1.204$
empirical risk (mean)			$2/5 = 0.40$	$1.39/5 = 0.278$	$3.786/5 = 0.757$

The three numbers measure different things. The error rate counts mistakes only. The cross-entropy also charges example 2, which is correct but hesitant, and it charges the confident mistake of example 4 (1.609) more than the milder mistake of example 5 (1.204); a completely undecided prediction, $g = 0.5$, would have cost $-\ln 0.5 = 0.693$ whatever the label. In bits

the cross-entropies are 0.152, 0.737, 0.515, 2.322, 1.737: the two wrong examples cost more than one bit each, as the margin argument above requires. (In margin form the score is the log-odds $s_i = \ln \frac{g_i}{1-g_i}$ and $z_i = t_i s_i$; example 4 has $s = \ln 0.25 = -1.39$ and $t = +1$, so $z = -1.39 < 0$, and example 5 has $z = -\ln \frac{0.7}{0.3} = -0.85$.) A model with the same 0–1 error but probabilities closer to the true labels (more confident when right, less confident when wrong) would have a lower cross-entropy, which matters whenever the threshold will be moved later.

Precision, recall and the F-score. When one class is rare — a signal among much background — the error rate hides what matters. Count the four outcomes of a binary decision on a test set ("positive" = class 1):

	predicted 1	predicted 0
true 1	TP (true positives)	FN (false negatives)
true 0	FP (false positives)	TN (true negatives)

This table is the *confusion matrix*. *Precision* $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$ is the fraction of predicted positives that are real; *recall* $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$ is the fraction of real positives that are found. The *F-score* F_1 is their harmonic mean, $F_1 = 2\,\mathrm{prec}\cdot\mathrm{rec}/(\mathrm{prec}+\mathrm{rec}) = 2\,\mathrm{TP}/(2\,\mathrm{TP}+\mathrm{FP}+\mathrm{FN})$, which is small as soon as either of the two is small. Moving the decision threshold trades one for the other.

Worked example — high accuracy, mediocre recall

On 1000 test events with 60 true positives a classifier gives TP = 40, FN = 20, FP = 10, TN = 930. Accuracy $= (40+930)/1000 = 0.97$. Precision $= 40/50 = 0.80$, recall $= 40/60 = 0.667$ and $F_1 = 2\cdot 0.8\cdot 0.667/(0.8+0.667) = 80/110 = 0.727$. The accuracy looks excellent, but a third of the positives are missed — and the trivial rule "always predict 0" already reaches accuracy $940/1000 = 0.94$ with recall 0 and an undefined precision.

In physics — efficiency, rejection and the ROC curve

A trigger or tagger outputs a score $g(\mathbf{x})$, and the analysis keeps events with $g(\mathbf{x}) > \tau$. The *signal efficiency* is the fraction of true signal events kept — the recall — and the *background rejection* the fraction of background events removed; the *purity* of the selected sample is the precision. Lowering τ raises the efficiency and lowers the rejection; the curve traced by all thresholds is the ROC (receiver operating characteristic) curve. The best threshold is fixed by the physics goal — for instance, the expected discovery significance — not by the training loss. Classifiers in physics are therefore typically trained with cross-entropy and then judged at the working point, or over the part of the ROC curve, that the analysis needs.

Modern connection — training loss versus what we want

Language models are pre-trained to minimise next-token cross-entropy, yet they are judged on answering questions, writing code and following instructions. Besides supervised instruction tuning, one of the main tools for closing that gap is fine-tuning on human preferences: a second model learns which of two answers people prefer, and the language model is optimised for that learned reward [9]. It is the hierarchy above, climbed one level at a time.

3 Expected risk and empirical risk

Assume, as in Note 1, that examples are i.i.d. draws from an unknown $P(\mathbf{X}, Y)$. The *expected risk* (generalisation error) of a fixed predictor f is its average loss on future data,

$$R(f) = \mathbb{E}_{(\mathbf{X},Y)\sim P}\big[L(f(\mathbf{X}), Y)\big],$$

and its *empirical risk* on a finite set D is

$$\hat{R}(f, D) = \frac{1}{|D|} \sum_{(\mathbf{x},y)\in D} L(f(\mathbf{x}), y).$$

We cannot compute $R(f)$, because P is unknown, but if f was chosen *without using* D, then $\hat{R}(f, D)$ is an honest estimate of it. The next subsection makes "honest" precise.

3.1 The test-set estimate: unbiased, with a computable error bar

Let $D_{\text{test}} = \{(\mathbf{x}_j, y_j)\}_{j=1}^{n'}$ be drawn i.i.d. from P, independently of everything used to build f. (Throughout this note n' is the number of test or held-out examples, also written n_{test} or n_{valid} when the role of the set matters; m keeps its course-wide meaning, the number of classes.) Write $L_j = L(f(\mathbf{x}_j), y_j)$. Because f is fixed with respect to D_{test}, the L_j are i.i.d. copies of the random variable $L(f(\mathbf{X}), Y)$, with mean $R(f)$ and some variance σ_L^2.

Mean. By linearity of expectation,

$$\mathbb{E}\big[\hat{R}(f, D_{\text{test}})\big] = \mathbb{E}\Big[\frac{1}{n'} \sum_{j=1}^{n'} L_j\Big] = \frac{1}{n'} \sum_{j=1}^{n'} \mathbb{E}[L_j] = \frac{1}{n'} \cdot n'\, R(f) = R(f).$$

The estimate is *unbiased*: on average over test sets it neither over- nor underestimates.

Variance. Independent terms have zero covariance, so the variance of the sum is the sum of the variances:

$$\operatorname{Var}\big[\hat{R}(f, D_{\text{test}})\big] = \frac{1}{(n')^2} \operatorname{Var}\Big[\sum_j L_j\Big] = \frac{1}{(n')^2} \sum_j \operatorname{Var}[L_j] = \frac{\sigma_L^2}{n'}.$$

The *standard error* is $\sigma_L/\sqrt{n'}$: to halve it, quadruple the test set. As $n' \to \infty$ the estimate converges to $R(f)$ (law of large numbers), and for moderate n' it is approximately Gaussian (central limit theorem), so $\hat{R} \pm 2\,\sigma_L/\sqrt{n'}$ is roughly a 95% interval.

The 0–1 loss. If f errs with probability $p = R(f)$, then $L_j \in \{0, 1\}$ is a Bernoulli variable. Since $L_j^2 = L_j$, $\sigma_L^2 = \mathbb{E}[L_j^2] - (\mathbb{E}L_j)^2 = p - p^2 = p(1-p)$, and the standard error of a test error is

$$\text{SE} = \sqrt{\frac{p(1-p)}{n'}} \approx \sqrt{\frac{\hat{p}(1-\hat{p})}{n'}}. \tag{1}$$

With $p = 10\%$ and $n' = 1000$ test examples that is about ± 0.95 percentage points: a difference of this size between two models can be produced by test-set noise alone. Requiring $2\,\text{SE} \le \delta$ gives the test size needed to measure *one* error rate to within $\pm\delta$: $n' \ge 4p(1-p)/\delta^2$, which for $p = 0.1$ and $\delta = 1$ point is 3600. To resolve a *difference* δ between two models evaluated on independent test sets the variance doubles, so $n' \ge 8p(1-p)/\delta^2 = 7200$; evaluating both on the same examples (next example) usually needs far fewer.

Worked example — is model B really better than model A

Two classifiers are evaluated on the *same* $n' = 1000$ test examples. A makes 80 errors ($\hat{p}_A = 8.0\%$), B makes 65 ($\hat{p}_B = 6.5\%$). From (1), $\mathrm{SE}_A = \sqrt{0.08 \cdot 0.92/1000} = 0.86\%$ and $\mathrm{SE}_B = 0.78\%$.
Naive comparison. Treating the two estimates as independent, the difference 1.5 points has standard error $\sqrt{0.86^2 + 0.78^2} = 1.16$ points: only 1.3 standard errors, not convincing.
Paired comparison. Both models saw the same examples, so compare them example by example. Suppose 45 examples are missed by both, 35 only by A and 20 only by B. Let $\Delta_j = L_{A,j} - L_{B,j} \in \{-1, 0, 1\}$ be the difference on example j; then $\hat{p}_A - \hat{p}_B = \bar{\Delta} = \frac{1}{n'} \sum_j \Delta_j = (35 - 20)/1000 = 0.015$. The Δ_j are i.i.d., so by the same argument as for $\hat{R}$ above, $\mathrm{Var}(\bar{\Delta}) = \mathrm{Var}(\Delta)/n'$. To estimate $\mathrm{Var}(\Delta) = \mathbb{E}[\Delta^2] - (\mathbb{E}\Delta)^2$, use the sample mean of the squares, $\overline{\Delta^2} = \frac{1}{n'} \sum_j \Delta_j^2$. Since $\Delta_j^2 = 1$ exactly when one model is wrong and the other right,

$$\widehat{\mathrm{Var}}(\Delta) = \overline{\Delta^2} - \bar{\Delta}^2 = \frac{35 + 20}{1000} - 0.015^2 = 0.0548, \qquad \mathrm{SE}(\bar{\Delta}) = \sqrt{0.0548/1000} = 0.74 \text{ points.}$$

(Dividing by $n' - 1$ instead of n', as in Algorithm 2, makes no visible difference at $n' = 1000$.) The difference is now 2.0 standard errors — right at the rough 95% threshold above: borderline but real evidence, where the naive comparison (1.3 standard errors) showed nothing. The examples both models get wrong (or right) have $\Delta_j = 0$ and carry no information about the difference; pairing removes their contribution to the noise. Always compare models on the same test set, and compare them pairwise.

3.2 Empirical risk minimisation

Since the expected risk is out of reach, learning minimises its estimate on the training set,

$$\hat{f} = \arg\min_{f \in \mathcal{F}} \hat{R}(f, D_{\text{train}}).$$

This principle is *empirical risk minimisation* (ERM), and it is the training phase of almost every method in the course. Algorithm 1 writes it generically; the three branches correspond to the three ways the optimisation ingredient of Section 1 is realised in practice.

Algorithm 1 Empirical risk minimisation (generic)

Require: family $\{f_\theta\}$, loss L, training set $D_{\text{train}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, optional penalty $\lambda\,\Omega(\theta)$
▷ the penalty is explained in Section 7.3; $\lambda = 0$ means no penalty
Ensure: fitted parameters $\hat{\theta}$

```
function ERM(D_train)
    J(θ) ← (1/n) Σ_{i=1}^n L(f_θ(x_i), y_i) + λ Ω(θ)        ▷ training objective
    if ∇J(θ) = 0 can be solved in closed form then
        θ̂ ← its solution                                    ▷ mean, least squares (Note 8)
    else if the family is finite, {θ_1, ..., θ_M} then
        θ̂ ← arg min_{1≤j≤M} J(θ_j)                          ▷ enumerate every candidate
    else
        θ̂ ← an iterative minimiser of J, e.g. gradient descent   ▷ Note 7
    end if
    return θ̂
end function
```

Running time — empirical risk minimisation

Let c be the cost of one prediction $f_\theta(\mathbf{x})$ ($c = O(d)$ for a linear model in d features) and q the number of parameters. *One evaluation of J*: $O(nc)$, one prediction and one loss per example. *Finite family*: $O(Mnc)$, every candidate on every example. *Iterative*: $O(N_{\text{ep}}\, nc)$ for N_{ep} full passes (epochs), since a gradient costs about as much as the objective itself (Note 7); stochastic gradients replace n by a mini-batch size. *Closed form*: problem-specific, e.g. $O(nd^2 + d^3)$ for least squares, the d^3 coming from solving a $d \times d$ linear system (Note 8). *Prediction*: c per query. *Memory*: $O(q)$ for the model plus the $O(nd)$ training data during training.

3.3 Why training error is optimistic

Now $\hat{f}$ *was* chosen using D_{train}, specifically to make $\hat{R}(\hat{f}, D_{\text{train}})$ small, so the argument of Section 3.1 no longer applies: the terms $L(\hat{f}(\mathbf{x}_i), y_i)$ are not independent of $\hat{f}$. The bias always goes the same way. Let $f_{\mathscr{F}} = \arg\min_{f \in \mathscr{F}} R(f)$ be the best member of the family; it is a fixed function, so its training error is unbiased. Then, for every training set,

$$\hat{R}(\hat{f}, D_{\text{train}}) \le \hat{R}(f_{\mathscr{F}}, D_{\text{train}}) \quad (\hat{f} \text{ minimises } \hat{R}) \qquad \text{and} \qquad R(f_{\mathscr{F}}) \le R(\hat{f}) \quad (f_{\mathscr{F}} \text{ minimises } R).$$

Taking expectations over D_{train} and using $\mathbb{E}[\hat{R}(f_{\mathscr{F}}, D_{\text{train}})] = R(f_{\mathscr{F}})$,

$$\mathbb{E}\big[\hat{R}(\hat{f}, D_{\text{train}})\big] \;\le\; R(f_{\mathscr{F}}) \;\le\; \mathbb{E}\big[R(\hat{f})\big].$$

Here $R(\hat{f})$ is itself a random number, because $\hat{f}$ depends on the random training set; the outer expectation averages over training sets. On average the training error of the trained model is below its true risk — often drastically, as the histogram classifier of Note 3 showed (5.2% training error, 34.6% test error).

Worked example — the optimism of the sample mean

Fit the constant predictor $\hat{b} = \bar{y}$ to n i.i.d. targets with mean μ and variance σ^2, with squared loss. Using $y_i - \bar{y} = (y_i - \mu) - (\bar{y} - \mu)$ and $\sum_i (y_i - \mu) = n(\bar{y} - \mu)$,

$$\sum_i (y_i - \bar{y})^2 = \sum_i (y_i - \mu)^2 - 2(\bar{y} - \mu)\, n(\bar{y} - \mu) + n(\bar{y} - \mu)^2 = \sum_i (y_i - \mu)^2 - n(\bar{y} - \mu)^2.$$

With $\mathbb{E}[(y_i - \mu)^2] = \sigma^2$ and $\mathbb{E}[(\bar{y} - \mu)^2] = \operatorname{Var}(\bar{y}) = \sigma^2/n$, the expected sum is $\mathbb{E}[\sum_i (y_i - \bar{y})^2] = n\sigma^2 - n \cdot \sigma^2/n = n\sigma^2 - \sigma^2$, so

$$\mathbb{E}[\text{training error}] = \frac{1}{n}\big(n\sigma^2 - \sigma^2\big) = \Big(1 - \frac{1}{n}\Big)\sigma^2.$$

For a new target Y, independent of the training set, the cross term vanishes and

$$\mathbb{E}\big[(Y - \bar{y})^2\big] = \mathbb{E}\big[(Y - \mu)^2\big] + \mathbb{E}\big[(\bar{y} - \mu)^2\big] = \Big(1 + \frac{1}{n}\Big)\sigma^2.$$

With $n = 5$ and $\sigma^2 = 1$: training error 0.8 on average, true risk 1.2 (a simulation over 200 000 training sets gives 0.798 and 1.199). The gap $2\sigma^2/n$ is the price of one fitted parameter. For least squares with q fitted coefficients, with the error measured on new targets drawn at the same inputs $\mathbf{x}_1, \dots, \mathbf{x}_n$, the same calculation gives a gap of $2q\sigma^2/n$ [3]: optimism grows with capacity and shrinks with data.

Never evaluate on data used for choosing $\hat{R}(f, D)$ is an honest estimate of $R(f)$ only if D played no part in choosing f — not in fitting the parameters, not in choosing hyperparameters, not in pre-processing, not in deciding which model to report.

Learning therefore has two phases. In the *training phase* ERM fits $\hat{f}$; memorising D_{train} is not the goal. In the *test phase* $\hat{f}$ is applied to new examples, and its risk is estimated by $\hat{R}(\hat{f}, D_{\text{test}})$ on a set never used before. That estimate is honest when D_{test} played no part in building $\hat{f}$, and precise when D_{test} is large enough (standard error $\sigma_L/\sqrt{n'}$, Section 3.1).

4 Estimating generalisation

Throughout this section A denotes a learning algorithm with fixed hyperparameters: given a training set it returns a predictor. $T_A(n)$ is its training cost on n examples and c the cost of one prediction.

4.1 Hold-out

Shuffle the data and split it: train on one part, report the empirical risk on the other (Algorithm 2). It is simple and correct; its two weaknesses with little data — held-out examples are not used for training, and the standard error $\sigma_L/\sqrt{n'}$ is large — are compared with the other protocols in the table at the end of this section. A common compromise holds out 20–30%.

Algorithm 2 Hold-out evaluation

Require: data D of size n, algorithm A, held-out fraction ρ (e.g. 0.2), with $\lceil \rho n \rceil \geq 2$
Ensure: estimate $\hat{R}$ of the risk of $A(D_{\text{train}})$ and its standard error

function HoldOut(D, A, ρ)
  shuffle D ▷ or split by group or time, Section 4.4
  $n' \leftarrow \lceil \rho n \rceil$; $D_{\text{test}} \leftarrow$ first n' examples; $D_{\text{train}} \leftarrow$ the rest
  $\hat{f} \leftarrow A(D_{\text{train}})$
  for $j = 1, \dots, n'$ **do**
    $L_j \leftarrow L(\hat{f}(\mathbf{x}_j), y_j)$ ▷ $(\mathbf{x}_j, y_j) \in D_{\text{test}}$
  end for
  $\hat{R} \leftarrow \frac{1}{n'} \sum_j L_j$; $s \leftarrow \sqrt{\frac{1}{n'-1} \sum_j (L_j - \hat{R})^2}$
  return $\hat{f}$, $\hat{R}$, $s/\sqrt{n'}$
end function

4.2 k-fold cross-validation

Split the data into k blocks (folds) of roughly equal size.[1] For $j = 1, \dots, k$, train on all blocks except j and compute the loss on block j; average the results (Algorithm 3, Figure 2, left). Every example is used once for evaluation and $k - 1$ times for training. Typical values are $k = 5$ or 10.

Cross-validation estimates the performance of the *procedure* A trained on a fraction $(k-1)/k$ of the data, not of one particular fitted function: the k fitted predictors are different. Because each is trained on fewer examples than the final model will be, the estimate is slightly pessimistic; larger k reduces this bias. The k fold errors are not independent (the training sets overlap), so the sample standard deviation of $e_1, \dots, e_k$ divided by $\sqrt{k}$ is only a rough standard error, but it is still useful for judging

[1] Both uses of the letter are standard: k counts the folds of k-fold cross-validation and the neighbours of k-NN. In this note k means folds in Sections 4 and 7, except in the name "k-NN"; in Section 5 and in questions about k-NN it means neighbours.

whether two settings differ (the one-standard-error rule, Section 7).

Algorithm 3 k-fold cross-validation

Require: data D of size n, algorithm A, number of folds k
Ensure: cross-validation estimate CV and the fold errors $e_1, \dots, e_k$

function CrossValidate(D, A, k)
  assign each example (or each whole group) at random to one of k folds $B_1, \dots, B_k$ of near-equal size
  for $j = 1, \dots, k$ **do**
    $\hat{f}_j \leftarrow A(D \setminus B_j)$ ▷ train without fold j
    $e_j \leftarrow \hat{R}(\hat{f}_j, B_j)$ ▷ evaluate on fold j
  end for
  return CV $= \sum_j |B_j| e_j / n$ and $(e_1, \dots, e_k)$
end function

4.3 Leave-one-out

The extreme case $k = n$ is *leave-one-out* cross-validation (LOO; the same leave-one-out idea applied to the bias or variance of a statistic is the jackknife of statistics): each example is predicted by a model trained on all the others (Algorithm 4). It wastes almost no data and has almost no pessimistic bias, but it needs n trainings, and its estimate can have a high variance because the n training sets are nearly identical. Two important cases are cheap. *Memory-based methods* (k-NN, Parzen windows, Note 5) have no training: leave-one-out only means excluding the point itself from its own neighbourhood, one pass over all pairwise distances, $O(n^2 d)$. *Linear smoothers* — methods whose predictions are fixed linear combinations of the training targets, such as the constant predictor and the least-squares and ridge regression of Note 8 — have an exact shortcut that obtains all n leave-one-out residuals from a single fit [3]; the constant predictor below is the simplest case.

Algorithm 4 Leave-one-out cross-validation

Require: data $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, algorithm A
Ensure: leave-one-out estimate LOO

function LeaveOneOut(D, A)
  for $i = 1, \dots, n$ **do**
    $\hat{f}_{-i} \leftarrow A(D \setminus \{(\mathbf{x}_i, y_i)\})$
    $L_i \leftarrow L(\hat{f}_{-i}(\mathbf{x}_i), y_i)$
  end for
  return LOO $= \frac{1}{n} \sum_i L_i$
end function

The LOO shortcut for the constant predictor. Removing y_i changes the mean to $\bar{y}_{-i} = (n\bar{y} - y_i)/(n-1)$, so the left-out residual is

$$y_i - \bar{y}_{-i} = \frac{(n-1)y_i - n\bar{y} + y_i}{n-1} = \frac{n}{n-1}(y_i - \bar{y}).$$

Every LOO residual is the ordinary residual inflated by $n/(n-1)$, and LOO $= \left(\frac{n}{n-1}\right)^2 \times$ training error: one fit instead of n.

Worked example — 3-fold cross-validation by hand

Six examples (x, y): $(1, 2), (2, 3), (3, 3), (4, 6), (5, 5), (6, 7)$. Compare two algorithms with squared loss: the constant predictor (predict the mean of the training targets) and 1-NN regression

(predict the target of the nearest training input, Note 5). For readability the folds are consecutive pairs; in practice they are chosen at random.

fold	validation x (y)	training y	constant: $\hat{y}$; errors	1-NN: neighbour x; errors
1	1, 2 (2, 3)	3, 6, 5, 7	5.25: 10.5625, 5.0625	$x = 3, 3$: 1, 0
2	3, 4 (3, 6)	2, 3, 5, 7	4.25: 1.5625, 3.0625	$x = 2, 5$: 0, 1
3	5, 6 (5, 7)	2, 3, 3, 6	3.5: 2.25, 12.25	$x = 4, 4$: 1, 1

Fold mean squared errors: constant 7.8125, 2.3125, 7.25; 1-NN 0.5, 0.5, 1.0. Averaging over the folds (all folds have the same size),

$$\mathrm{CV}_{\text{const}} = \frac{7.8125 + 2.3125 + 7.25}{3} = 5.79, \qquad \mathrm{CV}_{\text{1-NN}} = \frac{0.5 + 0.5 + 1.0}{3} = 0.67.$$

1-NN wins clearly: the targets follow a trend that a constant cannot represent (underfitting). Note that the *training* errors would not have told us this honestly: 1-NN has training error 0 by construction, whatever the data. For the constant predictor the training error is 3.22 (the mean is $26/6 = 4.33$), and the LOO shortcut gives LOO $= (6/5)^2 \times 3.22 = 4.64$ — smaller than the 3-fold value 5.79. Each LOO model is trained on five examples instead of four, and our folds are consecutive pairs: folds 1 and 3 remove the smallest or the largest targets, so the mean of the remaining four lies far from the left-out values.

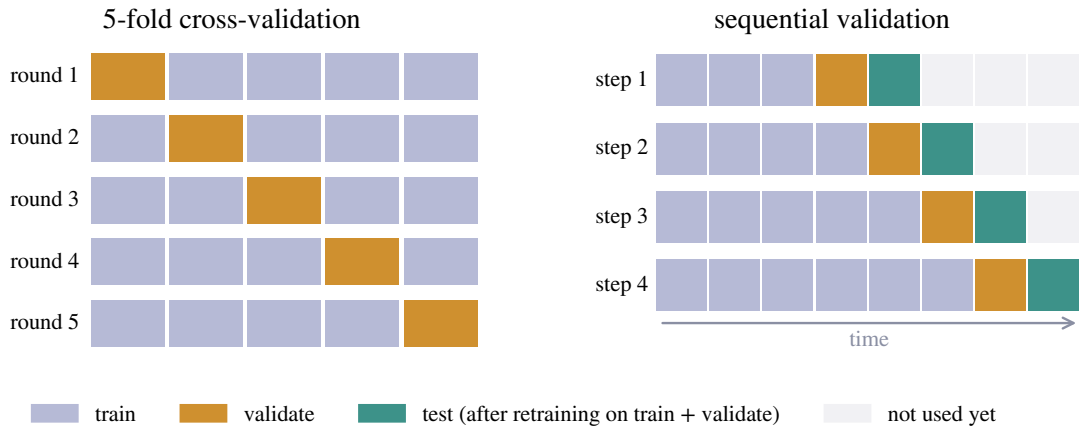


Figure 2. Left: in 5-fold cross-validation each block is the validation set once. Right: sequential validation for time-ordered data; at every step the candidates are trained on the past and compared on the block that follows, the winner is retrained on the past and that block, and it is tested on the next block (Algorithm 5).

4.4 Splits that respect the structure of the data

Shuffling assumes i.i.d. examples. When that fails (Note 1, Section 2), the split must follow the dependence:

- *Groups*: all measurements of the same galaxy, patient, sample or detector run go into the same block, or the model is tested on objects it has already seen.
- *Time*: for forecasting, train on the past and validate on the block that follows, then roll the window forward (*sequential validation*, Algorithm 5 and Figure 2, right). A random split would let the model learn from the future. When hyperparameters are chosen as well, each step has three parts: choose the setting on block B_t with models trained on the earlier blocks, retrain the winner on everything up to and including B_t, and report its error on B_{t+1}. This is Algorithm 8

of Section 7 with time-ordered training, validation and test sets, repeated as the origin rolls forward.

- *Simulation and reality*: a model trained and validated on simulation must still be tested on real data, or on a control region of it, before its performance is believed.

Algorithm 5 Sequential (rolling-origin) validation for time-ordered data

Require: time blocks $B_1, \dots, B_T$, algorithm A_γ with candidate settings Γ (G of them), initial blocks s, window w ($w = \infty$: expanding)
Ensure: test errors $e_{s+2}, \dots, e_T$ and their mean

function SequentialValidation($B_1, \dots, B_T, A, \Gamma, s, w$)
  for $t = s+1, \dots, T-1$ **do**
    $D_{\text{past}} \leftarrow B_{\max(1,\,t-w)} \cup \cdots \cup B_{t-1}$ ▷ the last w blocks, or all of the past
    for each $\gamma \in \Gamma$ **do**
      $v_\gamma \leftarrow \hat{R}(A_\gamma(D_{\text{past}}), B_t)$ ▷ validate on the next block
    end for
    $\gamma_t^* \leftarrow \arg\min_\gamma v_\gamma$; $\hat{f}_t \leftarrow A_{\gamma_t^*}(D_{\text{past}} \cup B_t)$ ▷ retrain, now including B_t
    $e_{t+1} \leftarrow \hat{R}(\hat{f}_t, B_{t+1})$ ▷ test on the block after
  end for
  return $(e_{s+2}, \dots, e_T)$ and their mean
end function

With a single setting ($G = 1$) nothing is chosen, so no validation block is needed: train on the past and evaluate directly on the block that follows.

Errors e_t that grow with t reveal drift: the process is changing and the model must be retrained regularly. When neighbouring time points are strongly correlated, leave a gap between the training and validation blocks.

Worked example — sequential validation by hand

A yearly quantity takes the values $y = 10, 12, 13, 15, 16, 18$ in years 1–6 (one value per block, so that everything can be done by hand). Two forecasters compete: M predicts the *mean* of all past years, L the *last* year's value. Squared loss, $s = 2$, expanding window.

step	validate on	M: forecast; error	L: forecast; error	choice	test: forecast, y; error
$t = 3$	$y_3 = 13$	11; 4	12; 1	L	year 4: 13, 15; 4
$t = 4$	$y_4 = 15$	11.67; 11.1	13; 4	L	year 5: 15, 16; 1
$t = 5$	$y_5 = 16$	12.5; 12.25	15; 1	L	year 6: 16, 18; 4

After each choice L is "retrained" on the years up to and including the validation year, i.e. it forecasts that year's value. The honest test errors are 4, 1, 4, mean 3. The validation errors of M grow (4, 11.1, 12.25): the series drifts upwards, and an average over the whole past falls further and further behind.

4.5 Nested cross-validation

When the same small dataset must both select hyperparameters and estimate the error of the whole procedure, use two loops — *nested* or *double cross-validation* (Algorithm 6). We write γ for one hyperparameter setting (a single value or a tuple of values), A_γ for the algorithm run with it and Γ for the set of candidates; the letter λ is kept for a penalty strength (Section 7.3). The two loops are an outer cross-validation for the estimate and, inside each outer training set, an inner cross-validation for the selection. The outer test fold never influences any choice, so the outer estimate is honest for

the *procedure* "tune by inner CV, then train". The hyperparameters chosen in different outer folds may differ; that is expected, and their spread is itself informative.

Algorithm 6 Nested cross-validation

Require: data D, algorithm family A_γ, candidate settings Γ (G of them), outer folds K, inner folds k
Ensure: estimate of the risk of the whole tuning-and-training procedure

function NestedCV(D, A, Γ, K, k)
  split D at random into outer folds $C_1, \dots, C_K$
  for $j = 1, \dots, K$ **do**
    $D_j \leftarrow D \setminus C_j$ ▷ outer training set
    for each $\gamma \in \Gamma$ **do**
      $v_\gamma \leftarrow$ CrossValidate(D_j, A_γ, k) ▷ inner loop, Algorithm 3
    end for
    $\gamma_j^* \leftarrow \arg\min_\gamma v_\gamma$; $\hat{f}_j \leftarrow A_{\gamma_j^*}(D_j)$
    $e_j \leftarrow \hat{R}(\hat{f}_j, C_j)$ ▷ C_j untouched until now
  end for
  return $\frac{1}{K}\sum_j e_j$ ▷ final model: tune by CV on all of D, then refit
end function

Worked example — nested cross-validation on six points

Take the six points of the 3-fold example above as outer folds ($K = 3$, consecutive pairs). Inside each outer training set of four points, leave-one-out chooses between the constant predictor and 1-NN regression (when two neighbours are equally close, average their targets). For the constant, the LOO shortcut gives the inner score directly: on the targets 3, 6, 5, 7 of outer fold 1 the training error is $8.75/4 = 2.1875$, and $(4/3)^2 \times 2.1875 = 3.89$.

outer fold	inner training x	LOO, constant	LOO, 1-NN	chosen	outer-fold error
1 ($x = 1, 2$)	3, 4, 5, 6	3.89	4.81	constant	7.81
2 ($x = 3, 4$)	1, 2, 5, 6	6.56	2.50	1-NN	0.50
3 ($x = 5, 6$)	1, 2, 3, 4	4.00	3.125	1-NN	1.00

The outer errors are those of the 3-fold example for the chosen method, so the nested estimate is $(7.81 + 0.50 + 1.00)/3 = 3.10$. It is much larger than 0.67, the cross-validation error of the winner: with four points per inner set the selection itself is unreliable (outer fold 1 picks the worse method), and nested cross-validation charges the procedure for that.

Running time — evaluation protocols

Protocol	Training cost	Use it when	Watch out for
Hold-out (Alg. 2)	$T_A(n - n')$, one training	n large; training expensive	noisy if n' is small
k-fold (Alg. 3)	$k\,T_A\left(\frac{k-1}{k}n\right)$	n moderate; the default	groups straddling folds
Leave-one-out (Alg. 4)	$n\,T_A(n-1)$, or one fit with a shortcut	n small; k-NN, Parzen, linear smoothers	high variance; cost without a shortcut
Sequential (Alg. 5)	$(T - s - 1)(G + 1)$ trainings, each at most $T_A(n)$	time-ordered data	leakage across the time boundary
Nested (Alg. 6)	$K(Gk + 1)$ trainings, $+(Gk + 1)$ for the final model	small n; honest estimate of a tuned procedure	cost

Here n is the number of examples, $T_A(n)$ the cost of one training, c the cost of one prediction and G the number of hyperparameter settings. *Prediction*: every protocol predicts each example at most once per training, $O(nc)$ per round. *Memory*: $O(nd)$ for the data, $O(n)$ for the fold indices and one model at a time; nested CV also keeps G scores per outer fold. *Why the terms*: k-fold trains k times on all but one fold, so for T_A linear in n it costs about $k-1$ full trainings; naive leave-one-out of least squares, $T_A = O(nd^2 + d^3)$, costs $O(n^2d^2 + nd^3)$; sequential validation trains the G candidates and one refit at each of its $T - s - 1$ steps; nested CV runs a full inner search (Gk trainings) plus one refit in each of K outer folds — with $K = k = 5$ and $G = 20$, 505 trainings, about 8.4 hours at one minute each, plus 101 more to tune and refit the final model on all of D. All folds, steps and settings are independent, so on N_{proc} processors or GPUs the wall-clock time drops by up to a factor N_{proc}.

Whatever the protocol, the same three rules apply: every choice is made without the data used for the final estimate; the split follows the dependence structure of the data; and every reported error comes with an error bar.

5 Capacity, underfitting and overfitting

The *capacity* of a family $\mathcal{F}$ is, informally, how many different patterns it can fit. It usually grows with the number of free parameters, but what matters is the effective number of degrees of freedom: constraints, shared parameters or regularisation can make a large model behave like a small one. For k-NN, which has no fitted parameters at all, the effective number of parameters is about n/k (roughly one local average per neighbourhood of k points) [3]. For restricted settings capacity can be measured exactly. The *Vapnik–Chervonenkis (VC) dimension* of a family of classifiers is the largest number N such that *some* arrangement of N points can be given every one of the 2^N labellings by members of the family (it is enough that one arrangement works: three points on a line cannot be labelled $+, -, +$ by a straight line, but three points in general position can). For affine classifiers $f(\mathbf{x}) = \mathbf{1}\{\mathbf{w}^{\mathsf{T}}\mathbf{x} + b > 0\}$ in $\mathbb{R}^d$ it is $d + 1$. In the plane ($d = 2$), three points not on a line can be given all $2^3 = 8$ labellings by straight lines, but no four points can. We only check the square here: for its four corners, the labelling that puts diagonally opposite corners in the same class is impossible (the general proof for four points is longer).

Figure 3 shows the three regimes on 15 noisy points from $\sin(\pi x)$. A straight line cannot follow the curve (*underfitting*: high training and test error). A polynomial of degree 12 passes close to every point and oscillates wildly between them (*overfitting*: low training error, high test error). Degree 3 captures the trend and ignores the noise.

The same three regimes appear in classification, where capacity shows up as the flexibility of the decision boundary. Note 5, Section 1, shows k-NN boundaries for several k with their training and test errors, and Note 3, Section 2, shows histogram classifiers with coarser and finer cells. The boundary that generalises best does not have the fewest training errors (in Note 5, about 6% for $k = 15$ against 0% for $k = 1$). It can even make *more* training errors than an underfitting straight line: what it is judged on is the error on new data, and a boundary that bends to avoid every training error is following noise.

Repeating the experiment on 200 training sets of $n = 20$ points and taking the median errors gives Figure 4: training error decreases monotonically with the degree, while validation error first falls, reaches its minimum at degree 3 and then rises steeply. The optimal capacity depends on the amount of data — with more points, higher degrees become affordable. Occam's razor — among models that explain the data equally well, prefer the simpler one — is the qualitative version of this curve.

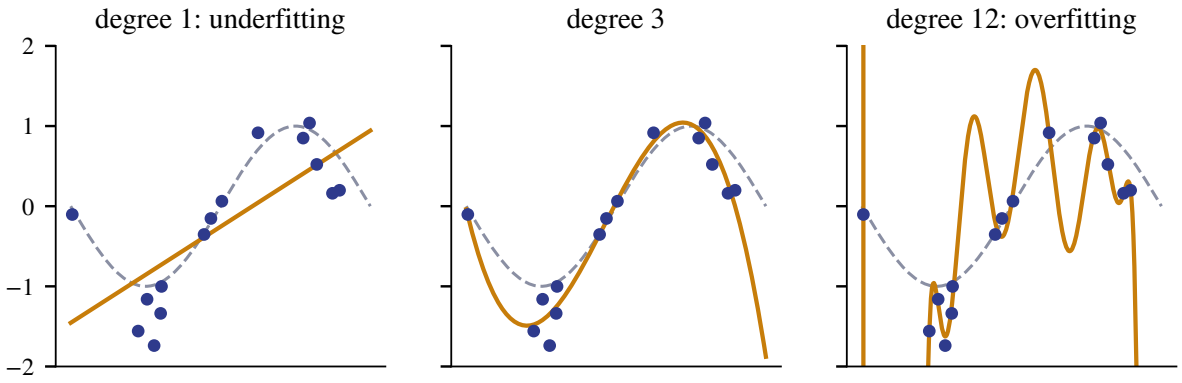


Figure 3. Least-squares polynomial fits to the same 15 points (dashed: the true function $\sin(\pi x)$).

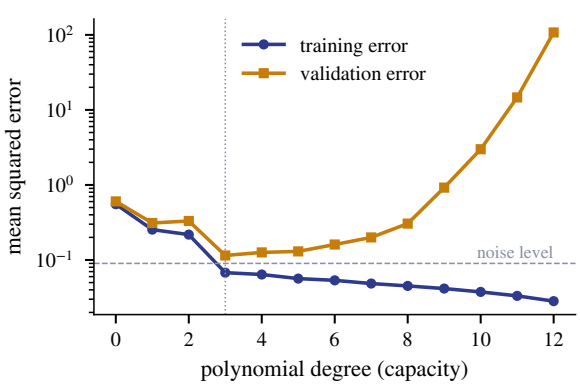


Figure 4. Median training and validation error over 200 training sets of $n = 20$ points. The dashed line is the noise variance 0.09, the smallest error any predictor can achieve.

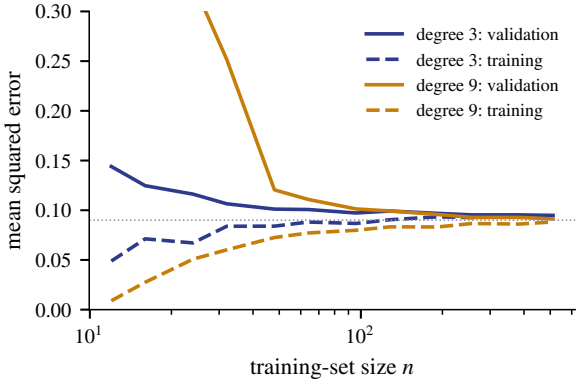


Figure 5. Learning curves: median error against training-set size for degrees 3 and 9. Training and validation error converge to the same limit; the more flexible model needs more data to get there.

Validation curves and learning curves. A plot of training and validation error against a hyperparameter at fixed n, such as Figure 4, is a *validation curve*; some texts also call it a learning curve. In this course a *learning curve* is the error against the training-set size n, as in Figure 5. Algorithm 7 computes both.

Running time — validation and learning curves

Validation curve: $\sum_g T_{A_{yg}}(n)$, one training per setting, plus $O(G(n+n_{\text{valid}})c)$ for the evaluations. *Learning curve*: $\sum_i T_A(n_i) \le r\,T_A(n)$; for sizes that double ($n_i = n/2^{r-i}$) and a training cost linear in n the sum is below $2\,T_A(n)$, about two full trainings. In practice each curve is averaged over several random splits (Figure 4 uses 200), which multiplies the cost. *Memory*: $O(nd)$ for the data and $O(G)$ or $O(r)$ numbers for the curves.

Reading a learning curve. Curves of training and validation error against n, as in Figure 5, diagnose what limits a model. If both errors are close to each other but high, the model underfits: more data will not help, a richer family will. If the training error is low and the validation error much higher, the model overfits: more data, a smaller family or stronger regularisation will help. The size of the gap at the current n, and how fast it is closing, tells whether collecting more data is worth the cost.

Algorithm 7 Validation curve and learning curve

Require: training set D_{train} of size n, validation set D_{valid}, algorithm A_γ

function ValidationCurve(settings $\gamma_1, \dots, \gamma_G$)
  for $g = 1, \dots, G$ **do**
    $\hat{f} \leftarrow A_{\gamma_g}(D_{\text{train}})$; record $\hat{R}(\hat{f}, D_{\text{train}})$ and $\hat{R}(\hat{f}, D_{\text{valid}})$
  end for
end function
function LearningCurve(setting γ, sizes $n_1 < n_2 < \cdots < n_r = n$)
  shuffle D_{train} once ▷ nested subsets: $D_1 \subset D_2 \subset \dots$
  for $i = 1, \dots, r$ **do**
    $D_i \leftarrow$ the first n_i examples; $\hat{f} \leftarrow A_\gamma(D_i)$
    record $\hat{R}(\hat{f}, D_i)$ and $\hat{R}(\hat{f}, D_{\text{valid}})$
  end for
end function

Worked example — counting capacity and reading two learning curves

Effective size of k-NN. With $n = 200$ training points, k-NN behaves like a model with about n/k parameters: 200 for $k = 1$ (every point its own region), 40 for $k = 5$, 10 for $k = 20$ and 1 for $k = 200$, where every query receives the same average — the constant predictor of Section 1. Increasing k therefore *lowers* the capacity, although nothing is "removed" from the model.
Two diagnoses. At $n = 1000$, model A has training error 0.30 and validation error 0.32, and both curves have been flat since $n = 300$. Model B has training error 0.02 and validation error 0.25, and the gap shrinks slowly as n grows. A underfits: the gap (0.02) is small, so more data cannot help; a richer family or better features can. B overfits: the gap (0.23) dominates, so more data, a smaller family or stronger regularisation will help. Both curves are useful only because the validation errors are computed on data the models never saw.

6 Where the error comes from

6.1 Approximation and estimation error

Let $f^* = \arg\min_f R(f)$ be the best possible predictor (its risk is the Bayes risk of Note 1, Section 4), $f_{\mathscr{F}} = \arg\min_{f \in \mathscr{F}} R(f)$ the best member of the family, and $\hat{f}$ the function returned by the algorithm. Adding and subtracting $R(f_{\mathscr{F}})$,

$$R(\hat{f}) - R(f^*) = \underbrace{R(f_{\mathscr{F}}) - R(f^*)}_{\text{approximation error}} + \underbrace{R(\hat{f}) - R(f_{\mathscr{F}})}_{\text{estimation error}}.$$

Both terms are non-negative: f^* is the best of all functions, and $f_{\mathscr{F}}$ is the best in $\mathscr{F}$, which contains $\hat{f}$. The approximation error depends only on the family and the problem; it shrinks (or stays the same) as $\mathscr{F}$ grows. The estimation error — the price of learning $f_{\mathscr{F}}$ from finitely many noisy examples — is random, usually grows with the size of $\mathscr{F}$ at fixed n, and shrinks with n.

Worked example — approximating a parabola with constants, lines and parabolas

Let X be uniform on $[-1, 1]$ and $Y = X^2 + \varepsilon$ with noise variance $\sigma^2 = 0.04$, squared loss. The Bayes predictor is $f^*(x) = x^2$ with risk 0.04, and $R(f) - R(f^*) = \mathbb{E}[(f(X) - X^2)^2]$ (Note 1, Section 4).

Constants. $\mathbb{E}[(b - X^2)^2]$ is minimised by $b = \mathbb{E}[X^2] = \frac{1}{2}\int_{-1}^{1} x^2\,dx = \frac{1}{3}$, and with $\mathbb{E}[X^4] = \frac{1}{2}\int_{-1}^{1} x^4\,dx = \frac{1}{5}$ the approximation error is $\mathrm{Var}(X^2) = \mathbb{E}[X^4] - (\mathbb{E}X^2)^2 = \frac{1}{5} - \frac{1}{9} = \frac{4}{45} = 0.089$. For the sample mean the estimation error is $\mathbb{E}[(\bar{y} - \frac{1}{3})^2] = \mathrm{Var}(Y)/n$, and $\mathrm{Var}(Y) = \mathrm{Var}(X^2) + \mathrm{Var}(\varepsilon) = 0.089 + 0.04$ because X and ε are independent.
Lines. Minimise $\mathbb{E}[(a + bX - X^2)^2]$ over a and b. Setting the two partial derivatives to zero,

$$\mathbb{E}[a + bX - X^2] = 0, \qquad \mathbb{E}[(a + bX - X^2)X] = 0,$$

gives $a = \mathbb{E}[X^2] - b\,\mathbb{E}[X]$ and, after substituting, $b = \mathrm{Cov}(X, X^2)/\mathrm{Var}(X)$. Here $\mathbb{E}[X] = 0$ and $\mathrm{Cov}(X, X^2) = \mathbb{E}[X^3] - \mathbb{E}[X]\mathbb{E}[X^2] = 0$ by symmetry, so $b = 0$ and $a = \frac{1}{3}$: the best line is the same constant, and the approximation error is *unchanged*, 0.089. (Without any algebra: X^2 is even in X, so a slope can only add error.) The extra parameter adds only estimation error.
Parabolas. f^* is in the family: approximation error 0.
Estimation errors from 10^5 simulated training sets (least-squares fits):

	approximation	estimation $n = 10$	excess $n = 10$	estimation $n = 100$	excess $n = 100$
constants	0.089	0.013	0.102	0.0013	0.090
lines	0.089	0.047	0.136	0.0032	0.092
parabolas	0	≈ 0.03	≈ 0.03	0.0012	0.0012

The "excess" columns are $R(\hat{f}) - R(f^*)$, approximation plus estimation; add the noise 0.04 for the expected risk itself. A bigger family is not automatically better: lines are worse than constants here. Parabolas, on the other hand, have a *smaller* estimation error than lines although they have more parameters: for lines the unexplained curvature acts like extra noise (residual variance $0.089 + 0.04 = 0.129$ instead of 0.04), and for large n the estimation error is roughly residual variance × number of parameters $/\,n$ — at $n = 100$, $0.129 \times 2/100 = 0.0026$ for lines and $0.04 \times 3/100 = 0.0012$ for parabolas. With the right family the estimation error dominates, and it falls roughly like $1/n$.

6.2 Bias and variance for squared loss

Let $y = f^*(\mathbf{x}) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$ and $\mathrm{Var}(\varepsilon) = \sigma^2$, and let $\hat{f}_D$ be trained on a random training set D. At a fixed $\mathbf{x}$, averaging over both D and the noise of a new observation,

$$\mathbb{E}\big[(y - \hat{f}_D(\mathbf{x}))^2\big] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{\big(f^*(\mathbf{x}) - \mathbb{E}_D[\hat{f}_D(\mathbf{x})]\big)^2}_{\text{bias}^2} + \underbrace{\mathrm{Var}_D\big(\hat{f}_D(\mathbf{x})\big)}_{\text{variance}}. \tag{2}$$

Derivation. Fix $\mathbf{x}$ and abbreviate $F = \hat{f}_D(\mathbf{x})$ (random through D), $\bar{F} = \mathbb{E}_D[F]$ and $f^* = f^*(\mathbf{x})$. The new noise ε is independent of D.

Step 1: separate the noise. Expanding the square,

$$\mathbb{E}\big[(f^* + \varepsilon - F)^2\big] = \mathbb{E}[\varepsilon^2] + 2\,\mathbb{E}\big[\varepsilon\,(f^* - F)\big] + \mathbb{E}\big[(f^* - F)^2\big].$$

By independence $\mathbb{E}[\varepsilon(f^* - F)] = \mathbb{E}[\varepsilon]\,\mathbb{E}[f^* - F] = 0$, and $\mathbb{E}[\varepsilon^2] = \sigma^2$. So the expected error is $\sigma^2 + \mathbb{E}[(f^* - F)^2]$, which is Note 1's decomposition with the trained model in place of a fixed one.

Step 2: separate bias and variance. Add and subtract $\bar{F}$:

$$\mathbb{E}\big[(f^* - F)^2\big] = \mathbb{E}\big[(f^* - \bar{F} + \bar{F} - F)^2\big] = (f^* - \bar{F})^2 + 2\,(f^* - \bar{F})\,\mathbb{E}[\bar{F} - F] + \mathbb{E}\big[(\bar{F} - F)^2\big].$$

The first term is a constant (it does not depend on D). The middle term vanishes because $\mathbb{E}[\bar{F} - F] = \bar{F} - \bar{F} = 0$. The last term is $\mathrm{Var}_D(F)$ by definition. Combining the two steps gives (2); averaging it over $\mathbf{x}$ gives the same decomposition of the risk.

Interpretation. Bias measures how far the *average* fitted function is from the truth; variance measures how much the fit changes from one training set to another. A richer family lowers bias and raises variance; more data lowers variance (Figure 5), which is why collecting more data is often the most effective way to reduce overfitting. Figure 6 computes both terms exactly for polynomial fits to 15 equally spaced points with noise $\sigma = 0.3$: degree 1 has squared bias 0.209 and variance 0.011; degree 3 balances them (0.006 and 0.021, total expected error 0.116 including the noise 0.09); degree 12 has negligible bias and variance 0.486. Bias roughly corresponds to approximation error and variance to estimation error, although the two decompositions are not identical.

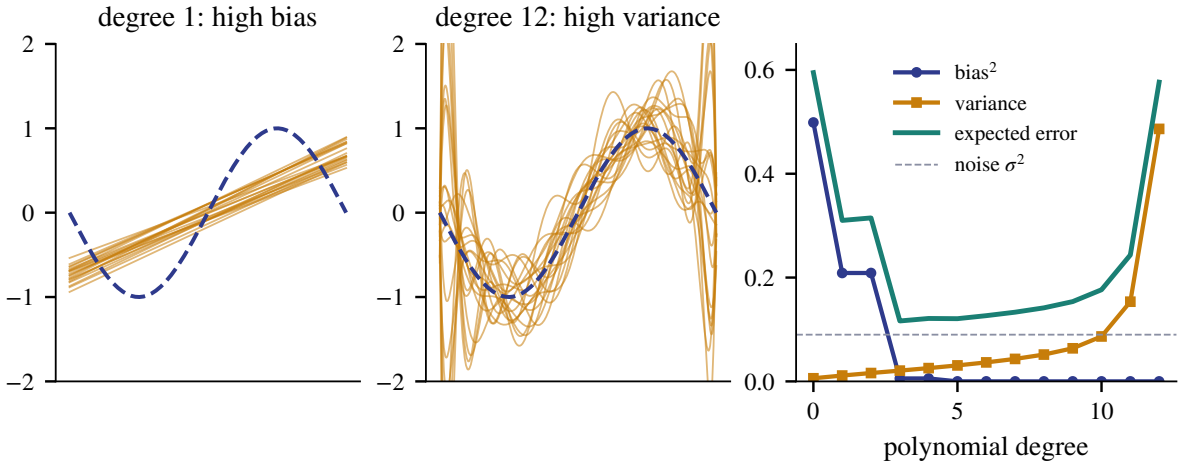


Figure 6. Bias and variance. Left and middle: 20 least-squares fits, each to a fresh noisy sample at the same 15 inputs (dashed: the truth). Right: squared bias and variance, averaged over $x \in [-1, 1]$, and their sum plus the noise.

Worked example — a biased estimator that beats the unbiased one

Estimate a mean μ from n observations with variance σ^2 by the *shrunk* mean $\hat{\mu}_a = a\,\bar{y}$, $0 \le a \le 1$. Since $\mathbb{E}[\bar{y}] = \mu$ and $\mathrm{Var}(\bar{y}) = \sigma^2/n$,

$$\text{bias} = \mathbb{E}[\hat{\mu}_a] - \mu = (a-1)\mu, \qquad \text{variance} = a^2\,\sigma^2/n.$$

The mean squared error of the estimator, $\mathbb{E}[(\hat{\mu}_a - \mu)^2]$, splits exactly as in Step 2 of the derivation above, with μ in place of f^* and $\hat{\mu}_a$ in place of F: it is bias^2 + variance. There is no noise term, because the estimate is compared with μ itself, not with a new noisy observation. Take $\mu = 2$, $\sigma^2 = 4$, $n = 4$, so $\sigma^2/n = 1$:

a	1	0.9	0.8	0.7	0.5	0
bias$^2 = (1-a)^2\mu^2$	0	0.04	0.16	0.36	1.00	4.00
variance $= a^2\sigma^2/n$	1.00	0.81	0.64	0.49	0.25	0
mean squared error	1.00	0.85	**0.80**	0.85	1.25	4.00

Setting the derivative of $(1-a)^2\mu^2 + a^2\sigma^2/n$ to zero, $-2(1-a)\mu^2 + 2a\sigma^2/n = 0$, gives $a^* = \mu^2/(\mu^2 + \sigma^2/n) = 4/5$: shrinking by 20% lowers the error from 1.00 to 0.80 (a simulation gives 0.996 and 0.795). For predicting a new observation the noise $\sigma^2 = 4$ is added to every

column. The catch is that a^* depends on the unknown μ; in practice the amount of shrinkage is a hyperparameter chosen on validation data. In fact $a\,\bar{y}$ is the minimiser of the regularised objective $\frac{1}{n}\sum_i (y_i - b)^2 + \lambda b^2$ with $a = 1/(1+\lambda)$ (Question B6), the one-parameter version of ridge regression (Note 8, Section 4).

7 Model selection

Parameters and hyperparameters. *Parameters* are what the training algorithm fits. *Hyperparameters* are set before training starts: a polynomial degree or a network architecture selects the family, a penalty strength λ says which of its members are preferred, a learning rate or a number of epochs steers the optimiser. As in Section 4, γ denotes one complete setting of the hyperparameters and λ only a penalty strength. Most of them control capacity, which is why they cannot be chosen on the training set: the training error would always favour the largest capacity.

Method	Hyperparameters	Parameters
Histogram (Note 3)	number and placement of bins	counts in each cell
Decision tree (Note 3)	maximum depth, minimum leaf size	split variables, thresholds, leaf values
k-NN (Note 5)	k, the distance	the stored training set
Parzen windows (Note 5)	kernel and its width	the stored training set
Gaussian Bayes classifier (Note 6)	shared or separate covariances, shrinkage	class means, covariances, priors
Polynomial, ridge, lasso regression (Note 8)	degree, λ, whether and how to scale features	coefficients
Logistic regression, SVM (Note 9)	λ or C; kernel and width (Note 10)	weights and bias
Neural network	architecture, learning rate, batch size, training time, λ	weights and biases

Worked example — sorting the choices of a real pipeline

A 5-NN classifier with Euclidean distance is trained on standardised features, using 20% of the data for validation.

- $k = 5$ and the Euclidean distance: *hyperparameters*, chosen on validation data.
- Whether to standardise at all: a *hyperparameter*. The means and standard deviations it uses: *parameters* — they are computed from data, so they must be computed from the training part only (Note 1, Section 5). Computing them on all data is leakage: the validation examples then shape the features the model sees, which tends to make the estimate optimistic (usually only slightly for standardisation, much more for steps such as feature selection).
- The stored training examples: the parameters of a memory-based method.
- The 20% validation fraction and the random seed of the split: *neither*; they belong to the evaluation protocol. The seed should not be tuned, and the variation across seeds is part of the error bar.
- The decision threshold of a probabilistic classifier: a hyperparameter if chosen to optimise a metric on validation data, a fixed rule ($\frac{1}{2}$) otherwise.

7.1 Selection with a training, a validation and a test set

Algorithm 8 is the basic meta-algorithm of the course: every candidate is trained on D_{train}, scored on D_{valid}, and only the winner is evaluated, once, on D_{test}.

Algorithm 8 Model selection with a training, a validation and a test set

Require: data D, candidate algorithms A, each with a set Γ_A of hyperparameter settings
Ensure: selected model $\hat{f}_{\text{sel}}$ and an honest estimate of its risk
1: **function** SELECTMODEL(D, $\{(A, \Gamma_A)\}$)
2: Shuffle (or split by group or time, Section 4.4) and divide D into D_{train}, D_{valid}, D_{test}
3: **for** each candidate algorithm A and each setting $\gamma \in \Gamma_A$ **do**
4: $\hat{f}_{A,\gamma} \leftarrow A_\gamma(D_{\text{train}})$ ▷ fit parameters
5: $e_{A,\gamma} \leftarrow \hat{R}(\hat{f}_{A,\gamma}, D_{\text{valid}})$ ▷ out-of-sample score
6: **end for**
7: $(A^*, \gamma^*) \leftarrow \arg\min e_{A,\gamma}$; $\hat{f}_{\text{sel}} \leftarrow \hat{f}_{A^*,\gamma^*}$
8: Optionally refit: $\hat{f}_{\text{sel}} \leftarrow A^*_{\gamma^*}(D_{\text{train}} \cup D_{\text{valid}})$
9: **return** $\hat{f}_{\text{sel}}$ and $\hat{R}(\hat{f}_{\text{sel}}, D_{\text{test}})$ ▷ D_{test} used once, at the very end
10: **end function**

Running time — model selection with train/validation/test

With G candidates (algorithms times settings): $\sum_{A,\gamma} T_{A_\gamma}(n_{\text{train}})$ for training, $O(G\, n_{\text{valid}}\, c)$ for validation, one optional refit and $O(n_{\text{test}}\, c)$ for the final test. Replacing the validation set by k-fold cross-validation multiplies the training cost by about k. Prediction: the selected model costs c per query; the selection adds nothing at prediction time. Memory: the data, $O(nd)$, plus the best model so far (or none, refitting at the end).

The validation error of the winner is itself optimistic — it is the minimum over many noisy estimates — which is why a separate test set is needed. The effect is easy to quantify. Suppose 20 settings are tried on 200 validation examples of a balanced binary problem, and *none* of them is better than guessing: each has true error 50% and, by (1), a standard error of 3.5 points. The smallest of the 20 validation errors is then on average 43.4% (by simulation, assuming the settings err independently of one another; correlated settings give a smaller but still positive optimism), an apparent improvement of more than six points that does not exist; with 200 settings the best reaches 40.3%. Cross-validation can replace the single validation set in line 5, which reduces the noise but does not remove this selection effect.

The one-standard-error rule. Validation curves are often flat near their minimum, and the differences between the best few settings are within noise. A common, conservative choice is to take the *simplest* model (lowest capacity, strongest regularisation) whose validation error is within one standard error of the best [3] (Algorithm 9). It needs no training beyond the cross-validation that produced the errors: $O(G)$ comparisons.

Algorithm 9 The one-standard-error rule

Require: settings $\gamma_1, \dots, \gamma_G$ ordered from simplest to most complex, their cross-validation errors CV_g and standard errors SE_g (Algorithm 3)
1: **function** ONESERULE($\text{CV}_{1..G}, \text{SE}_{1..G}$)
2: $g^* \leftarrow \arg\min_g \text{CV}_g$ ▷ the best setting
3: **return** the smallest g with $\text{CV}_g \le \text{CV}_{g^*} + \text{SE}_{g^*}$ ▷ simplest within one SE
4: **end function**

Worked example — choosing a polynomial degree with the one-standard-error rule

Ten-fold cross-validation on the training and validation data gives, for degrees 1–6 (mean ± standard error of the fold errors):

degree	1	2	3	4	5	6
CV ± SE	0.42 ± 0.03	0.15 ± 0.02	0.110 ± 0.015	0.105 ± 0.015	0.108 ± 0.02	0.12 ± 0.025

The minimum is at degree 4, 0.105, so the threshold is $0.105 + 0.015 = 0.120$. Degree 2 (0.15) is above it, degree 3 (0.110) below: the rule picks degree 3. Degrees 3–6 are indistinguishable within their error bars, and the simplest of them has the lowest variance. Refit degree 3 on all training and validation data and evaluate it once on the test set; that single number is what is reported.

7.2 Searching the hyperparameter space

With one hyperparameter one evaluates a list of values, usually on a logarithmic scale for scale-like quantities such as λ, a kernel width or a learning rate ($10^{-4}, 10^{-3}, \dots, 1$). With several hyperparameters there are two standard strategies (Algorithm 10). *Grid search* evaluates every combination of a few values per hyperparameter. *Random search* draws each setting at random from a chosen distribution and stops when the budget is spent.

Algorithm 10 Grid search and random search

Require: a scoring routine Score(γ) (validation error, or CrossValidate of A_γ); h hyperparameters

function GridSearch(value lists $V_1, \dots, V_h$)
  for each $\gamma \in V_1 \times V_2 \times \cdots \times V_h$ **do**
    $e_\gamma \leftarrow$ Score(γ)
  end for
  return $\arg\min_\gamma e_\gamma$ ▷ $|V_1| \cdots |V_h|$ evaluations
end function
function RandomSearch(distributions $\pi_1, \dots, \pi_h$, budget N_{tr})
  for $b = 1, \dots, N_{\text{tr}}$ **do**
    draw $\gamma_b = (\gamma_{b1}, \dots, \gamma_{bh})$ with $\gamma_{bj} \sim \pi_j$ independently
      ▷ e.g. log-uniform for a penalty λ and learning rates
    $e_b \leftarrow$ Score(γ_b)
  end for
  return γ_{b^*} with $b^* = \arg\min_b e_b$ ▷ N_{tr} evaluations, N_{tr} chosen freely
end function

Running time — grid search and random search

Each evaluation costs one training plus validation, or about k trainings with cross-validation. Grid search with v values per hyperparameter needs v^h evaluations: exponential in the number of hyperparameters (5 values for 4 hyperparameters is already 625 trainings). Random search needs exactly the budget N_{tr}, whatever h. Both are embarrassingly parallel. Prediction: c per query for the selected model; the search adds nothing. Memory: the list of scores, $O(v^h)$ or $O(N_{\text{tr}})$.

Why random search usually wins. Usually only a few hyperparameters matter much [2]. A 3×3 grid spends nine trainings on only three values of the important hyperparameter, because each value is repeated for every value of the unimportant one (Figure 7). Nine random trials give nine different values of every hyperparameter. A simple calculation shows how many trials are enough.

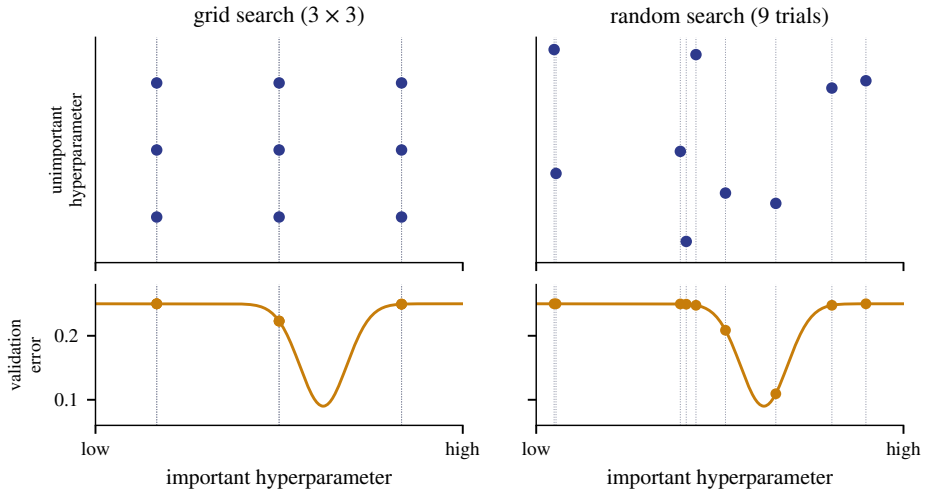


Figure 7. Nine trials when only the horizontal hyperparameter matters. Top: the trials in the plane of the two hyperparameters. Bottom: the validation error, which depends on the important hyperparameter only. The grid tests just three of its values and misses the dip (best error 0.223); random search tests nine and finds it (0.109). After the illustration of Bergstra and Bengio [2].

If a setting counts as "good" when it lies in the best 5% of the search space (in volume), each random trial misses with probability 0.95, and N_{tr} independent trials all miss with probability $0.95^{N_{\text{tr}}}$. So $P(\text{at least one good trial}) = 1 - 0.95^{N_{\text{tr}}}$: 0.37 for $N_{\text{tr}} = 9$, 0.64 for $N_{\text{tr}} = 20$ and 0.95 for $N_{\text{tr}} = 60$, independently of the number of hyperparameters. Grids remain fine for one or two hyperparameters, and as a final refinement around a good region. More elaborate methods (Bayesian optimisation, early termination of bad runs) build on the same scoring routine.

7.3 Regularisation

Instead of choosing among nested families, one can keep a rich family and add a preference for simple members,

$$\hat{f}_\lambda = \arg\min_{f \in \mathcal{F}} \; \hat{R}(f, D_{\text{train}}) + \lambda\, \Omega(f),$$

where Ω penalises complexity and $\lambda \geq 0$ is a hyperparameter. Weight decay ($\Omega = \|\mathbf{w}\|^2$) is the most common choice; it is studied for linear models as ridge regression (Note 8, Section 4), and the ℓ_1 penalty of the lasso (Note 8, Section 5) prefers sparse solutions. Stopping the optimisation early (Note 7, Section 7), injecting noise (dropout) and augmenting the data with transformed copies act as implicit regularisers. Shrinking the covariance estimate (Note 2, Section 6) is the same idea. Ω can even be the degree of a polynomial: minimising $\hat{R} + \lambda \cdot \text{degree}$ trades training error against degree, the soft analogue of choosing the degree on validation data. Penalised criteria of this kind are known as *complexity penalties*.

Worked example — the value of λ decides between two fits

Four points (x, y): $(-1, -2)$, $(0, 1)$, $(1, 2)$, $(2, 1)$, squared loss, and Ω the sum of squared coefficients *excluding* the intercept. Compare two candidates:

- A, the parabola $1 + 2x - x^2$, which passes through all four points: $\hat{R} = 0$, $\Omega = 2^2 + 1^2 = 5$;
- B, the least-squares line $y = x$, with residuals $-1, 1, 1, -1$: $\hat{R} = 1$, $\Omega = 1^2 = 1$.

The regularised objectives are $J_A(\lambda) = 0 + 5\lambda$ and $J_B(\lambda) = 1 + \lambda$. For $\lambda = 0.1$: $J_A = 0.5 < J_B = 1.1$, the interpolating parabola wins. For $\lambda = 0.5$: $J_A = 2.5 > J_B = 1.5$, the line wins. The preference

switches at $5\lambda = 1+\lambda$, i.e. $\lambda = 0.25$. The actual minimiser of J over all quadratics is neither A nor B (it is the ridge solution of Note 8, Section 4): it moves continuously from the interpolating parabola ($\lambda \to 0$) towards a flatter curve as λ grows, and reaches $J = 0.40$ at $\lambda = 0.1$ and $J = 1.13$ at $\lambda = 0.5$, below both candidates. Which choice generalises better is not decided by these four points; λ is chosen on validation data. Because Ω adds coefficients of terms of different units (x, x^2), its value depends on how x is scaled — one reason to standardise features before regularising (Note 8). With the degree penalty instead, $J_A = 2\lambda$ and $J_B = 1 + \lambda$, and the preference switches at $\lambda = 1$.

In physics — blind analysis

Particle and nuclear physicists guard against tuning an analysis towards the answer they expect by *blinding*: the signal region, or the final numerical result, stays hidden while selection criteria and corrections are fixed on simulation and control regions, and is revealed only once [6]. This is Algorithm 8 with the test set locked in a drawer — the discipline that keeps $\hat{R}(\hat{f}_{\text{sel}}, D_{\text{test}})$ honest.

Modern connection — how far does a benchmark number travel

Benchmark test sets reused by thousands of papers could slowly turn into validation sets. When a new ImageNet evaluation set was collected by the original procedure, accuracies fell by 11–14 percentage points, though the ranking of models was largely preserved; the authors traced the drop mainly to small differences between the old and new data, not to overfitting by reuse [10]. A test number holds for the distribution its examples came from. For language models trained on much of the public web a further danger is *contamination*: benchmark questions and answers may be in the training data. Fresh, private or time-stamped test sets guard against both.

8 Beyond the U-shaped curve

Modern neural networks often have far more parameters than training examples and can fit even randomly labelled data [11], yet they generalise well. Figure 8 shows a simple model with the same behaviour: least squares on p random nonlinear features of $n = 100$ training points. Each feature is a fixed random function $\phi_j(\mathbf{x}) = \max(0, \mathbf{v}_j^{\mathsf{T}}\mathbf{x})$, with the direction $\mathbf{v}_j$ drawn once at random and never trained (a ReLU feature), and the model is $f(\mathbf{x}) = \sum_{j=1}^{p} w_j \phi_j(\mathbf{x})$, linear in the weights $\mathbf{w}$. As p grows the test error first follows the classical U, then rises sharply as p approaches n — where the model can just interpolate the data and the fit is extremely sensitive to noise — and then *decreases again*. Beyond $p = n$ there are infinitely many interpolating solutions, and the algorithm returns the one whose weight vector has the smallest norm $\|\mathbf{w}\|$; as p grows, that implicit preference makes the fit smoother. In this example the test error at $p = 6000$ (0.134) is below the best value of the classical regime (0.141).

This *double descent* [1, 8] does not overturn the ideas of this note; it refines what "capacity" means. The number of parameters is a poor measure of it; the norm of the solution, set implicitly by the optimiser, matters more. Validation still decides. The other modern lesson concerns scale: for language models, test loss falls as a smooth power law in model size, data and compute over many orders of magnitude [5], and for a fixed compute budget model size and number of training tokens should grow together, at roughly twenty tokens per parameter [4]. Learning curves like Figure 5, measured on small models, are now used to plan the training of large ones.

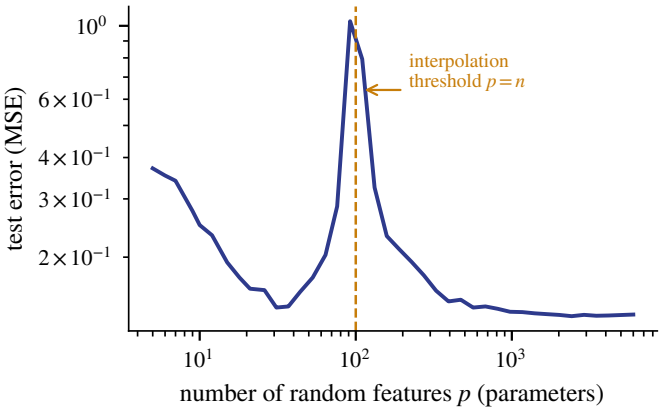


Figure 8. Double descent: median test error of minimum-norm least squares on p random ReLU features of 10-dimensional inputs, $n = 100$ training points, 20 repetitions.

Summary A learning algorithm is a family, a loss and an optimiser; the loss decides what "best" means, and continuous, convex surrogates that upper-bound the 0–1 error and have informative slopes make training possible; precision and recall describe rare classes better than the error rate. On data never used for any choice, the empirical risk is an unbiased estimate of the expected risk, with standard error $\sigma_L/\sqrt{n'}$ ($\sqrt{p(1-p)/n'}$ for an error rate); models are best compared pairwise on the same test set. Training minimises empirical risk, which underestimates the expected risk of the result. Honest estimates come from hold-out, k-fold, leave-one-out, sequential or nested validation, with splits that respect groups and time. Capacity trades approximation against estimation error, and bias against variance; hyperparameters and regularisation control it and are chosen on validation data — by random rather than grid search when there are several — with the test set used once. Over-parameterised models add a second descent of the test error, but not a new principle.

Questions and exercises

Questions are sorted by kind. Parts A and B are of the kind used in the midterm examination: they test understanding, interpretation and short calculations (at most simple derivatives), not long proofs. Parts C and D go further and are recommended for practice and for the project.

A. Conceptual questions

A1. Name the three ingredients (family, loss, optimiser) of (a) the constant predictor trained with squared loss, (b) least-squares line fitting, (c) a 1-NN classifier (hint: 1-NN only stores the data; ask what training error it achieves and what, if anything, is optimised). For which of them is the "optimiser" trivial, and why?

A2. Why do we not train classifiers by minimising the 0–1 error directly? State two properties a good surrogate loss should have, and explain why the hinge loss and the cross-entropy in bits have them.

A3. Two classifiers have the same test error but different test cross-entropies. What does the difference tell you? Which one would you prefer for a trigger whose threshold will be tuned later, and why?

A4. True or false, with a one-line justification each: (a) the error of a fixed classifier on a fresh test set is an unbiased estimate of its expected error; (b) the training error of a trained classifier is an unbiased estimate of its expected error; (c) quadrupling the test set halves the standard error of the test error.

A5. Two classifiers are evaluated on the same test set. Why does comparing them example by example detect smaller differences than comparing their two error rates as if they were independent? Which test examples carry no information about the difference?

A6. Explain in words why the training error of a fitted model is optimistic. Use the constant predictor (the sample mean) as your example: why is its training error smaller on average than its error on a new point?

A7. Explain why choosing k for k-NN by minimising the training error always favours $k = 1$ (it attains the smallest possible training error), and what the training error of 1-NN is (assume no two training points share the same input).

A8. Compare hold-out evaluation and k-fold cross-validation: what does each cost, what does each waste, and when is hold-out perfectly adequate?

A9. Cross-validation is said to estimate the performance of a *procedure*, not of a fitted function. Explain. Why is the k-fold estimate slightly pessimistic, and how does this change with k?

A10. Galaxy images come from 300 telescope pointings, several galaxies per pointing. Design the cross-validation splits, and explain what a naive random split would overestimate.

A11. You forecast the daily output of a solar plant. Why is a random k-fold split wrong here? Describe sequential validation and say what a growing validation error along the steps would indicate.

A12. A data set is standardised (mean and standard deviation computed on all of it) before 5-fold cross-validation. Why is this a leak, in which direction does it usually bias the estimate, and how should the pipeline be organised?

A13. Why does model selection need a validation set *and* a test set? What goes wrong when many settings are compared on one validation set and the best validation error is reported as the final result?

A14. Model A has training error 0.30 and validation error 0.32, both flat as n grows; model B has 0.02 and 0.25, with the gap closing slowly. Diagnose each, and say whether more data, a richer family or more regularisation would help.

A15. Why can straight lines in the plane not separate every labelling of the four corners of a square, although they can for any three points not on a line? What does this say about the capacity of linear classifiers in two dimensions?

A16. What happens to the bias and to the variance of a model when (a) the polynomial degree increases, (b) the number of training examples increases, (c) the regularisation strength λ increases? Explain each in a sentence.

A17. Define approximation error and estimation error. In the parabola example of Section 6, adding a slope to the constant predictor made things worse. Which term grew, and why did the other not shrink?

A18. Classify each item as parameter, hyperparameter or neither: the weights of a logistic regression; the regularisation constant C of an SVM; the bin width of a histogram; the counts in its bins; the random seed of the train/test split; the means used to standardise features; the number of training epochs of a neural network.

A19. Explain with a picture why random search usually finds better settings than grid search for the same number of trials. When is a grid still a good choice?

A20. State the one-standard-error rule. Why does it prefer the simpler model, and why is that reasonable when the validation curve is flat near its minimum?

A21. What problem does nested cross-validation solve, and what does it cost? Why may the outer folds select different hyperparameters, and why is that not a problem?

A22. Does double descent contradict the bias–variance trade-off? Explain what "capacity" means for a minimum-norm interpolating model, and why validation data still decide.

A23. A selection keeps 97% of events correctly classified on a sample in which only 6% of the events are signal. Why can this accuracy be misleading? Which two numbers would you report instead, and how are they related to the signal efficiency and the purity of the selected sample?

A24. In sequential validation with hyperparameter selection, why is the setting chosen on block B_t but the error reported on B_{t+1}? What would go wrong if the error on B_t itself were reported?

B. Short calculations

B1. A classifier outputs $g = 0.8, 0.4, 0.1, 0.9$ for four examples with labels $y = 1, 1, 0, 0$. Compute the empirical risk under the 0–1 loss (threshold $\frac{1}{2}$), the squared loss $(g - y)^2$ and the cross-entropy (natural logarithm). Which example contributes most to the squared loss and to the cross-entropy? Why does the 0–1 loss not distinguish examples 2 and 4?

B2. A classifier has 8.0% error on a test set of 500 examples and another has 7.0%. Estimate the standard error of each. Are the two classifiers distinguishable if their errors are treated as independent? On the same test set, the first is wrong and the second right on 25 examples, and the reverse on 20. Compute the paired standard error of the difference. For an error rate near 10%, how large a test set makes two standard errors of *one* classifier's error at most one percentage point? How large must two independent test sets be to resolve a one-point difference between two classifiers at two standard errors?

B3. Six examples (x, y): $(1, 1), (2, 3), (3, 2), (4, 5), (5, 4), (6, 6)$. Compute by hand the 3-fold cross-validation error (squared loss, folds $\{1, 2\}$, $\{3, 4\}$, $\{5, 6\}$ by x) of the constant predictor and of 1-NN regression. Which would you choose?

B4. For the targets $y = 0, 1, 1, 4, 9$ find the best constant under squared loss and under absolute loss, using the derivative of the training objective (for the absolute loss, the sign of $J'(b)$ between data points, as in Section 1), and compute both training risks for both constants.

B5. For the shrunk mean $a\,\bar{y}$ with $\mu = 1$, $\sigma^2 = 2$, $n = 2$, compute the squared bias, the variance and the mean squared error for $a = 1$ and $a = 0.5$. Find the optimal a by setting a derivative to zero.

B6. Show by differentiation that $J(b) = \frac{1}{n}\sum_i (y_i - b)^2 + \lambda b^2$ is minimised by $b = \bar{y}/(1 + \lambda)$. Then, for the one-parameter model $f(x) = wx$ with $J(w) = \frac{1}{n}\sum_i (wx_i - y_i)^2 + \lambda w^2$, show that $w = \sum_i x_i y_i / (\sum_i x_i^2 + n\lambda)$ and evaluate it for the data $(1, 1), (2, 3)$ with $\lambda = 0$ and $\lambda = 0.5$.

B7. Nested cross-validation uses $K = 10$ outer folds, $k = 5$ inner folds and a grid of 4×5 settings; one training takes three minutes. How many trainings does the outer estimate need, how many more are needed to tune and refit the final model on all the data, and how long does everything take on one machine?

B8. With random search, how many trials are needed to have at least a 90% chance that one trial lands in the best 5% of the search space? Does the answer depend on the number of hyperparameters?

B9. On 2000 test events a classifier gives TP $= 90$, FP $= 30$, FN $= 60$, TN $= 1820$. Compute its accuracy, precision, recall and F_1. What accuracy does the rule "always predict 0" reach on the same events?

B10. Five regularisation strengths, ordered from the strongest (simplest model) to the weakest, have cross-validation errors 0.30 ± 0.02, 0.22 ± 0.02, 0.20 ± 0.01, 0.19 ± 0.015 and 0.21 ± 0.02. Which setting has the smallest error, and which does the one-standard-error rule choose? Explain the difference in one sentence.

C. Derivations

C1. Prove the bias–variance decomposition (2), justifying why each cross term vanishes. Apply it to k-NN regression at a fixed $\mathbf{x}$ whose k neighbours (with fixed inputs) have independent noisy targets $y_{(j)} = f^*(\mathbf{x}_{(j)}) + \varepsilon_j$: show that the variance is σ^2/k and write the squared bias.

C2. For the constant predictor fitted to n i.i.d. targets, show that the expected training error is $(1 - 1/n)\sigma^2$ and the expected error on a new target is $(1 + 1/n)\sigma^2$. Then derive the leave-one-out identity $y_i - \bar{y}_{-i} = \frac{n}{n-1}(y_i - \bar{y})$ and show that the expected LOO error equals $(1 + \frac{1}{n-1})\sigma^2$, the expected risk of a model trained on $n - 1$ points.

C3. Prove that $\mathbb{E}[\hat{R}(\hat{f}, D_{\text{train}})] \le R(f_{\mathscr{F}}) \le \mathbb{E}[R(\hat{f})]$ for empirical risk minimisation over any family $\mathscr{F}$, and explain where the proof uses that $f_{\mathscr{F}}$ does not depend on the training set.

C4. For two models evaluated on the same test set of size n', show that $\operatorname{Var}(\hat{R}_A - \hat{R}_B) = \big(\operatorname{Var} L_A + \operatorname{Var} L_B - 2\operatorname{Cov}(L_A, L_B)\big)/n'$. Explain why pairing helps when the models tend to make the same mistakes, and recover the formula $\overline{\Delta^2} - \bar{\Delta}^2$ of the worked example.

C5. For the shrunk mean $a\,\bar{y}$, derive the optimal a^* and show that the minimal mean squared error is $\frac{\sigma^2}{n} \cdot \frac{\mu^2}{\mu^2 + \sigma^2/n}$, which is smaller than σ^2/n for every μ.

D. Programming

D1. Reproduce Figure 4 for $n = 20$, 100 and 1000 and report the degree that minimises the validation error in each case. Then fix degree 12 and $n = 20$ and add weight decay; plot validation error against λ on a logarithmic grid.

D2. Ten research groups each try 20 hyperparameter settings on the same public validation set and report the best. Simulate this with classifiers that guess at random on a balanced binary problem with 200 validation examples: plot the distribution of the reported errors, compare it with the errors of the same classifiers on a fresh test set, and explain the difference.

D3. Implement HoldOut, CrossValidate, LeaveOneOut and NestedCV in numpy for k-NN regression on synthetic data $y = \sin(\pi x) + 0.3\,\varepsilon$. Check your leave-one-out code against the

shortcut for the constant predictor, and time each protocol as n grows.

D4. Tune a two-hyperparameter model (for example, polynomial degree and λ, or k and the distance exponent of k-NN) with grid search and with random search using 9, 25 and 64 trials. Repeat over 50 random splits and compare the distributions of the best validation errors and of the test errors of the selected models.

Further reading. Hastie, Tibshirani and Friedman [3], chapter 7, for optimism, the bias–variance decomposition, cross-validation and the one-standard-error rule; Murphy [7], chapters 4–5; Bergstra and Bengio [2] for random search; for double descent, Belkin *et al.* [1].

References

[1] M. Belkin, D. Hsu, S. Ma and S. Mandal, Reconciling modern machine-learning practice and the classical bias–variance trade-off, *Proc. Natl. Acad. Sci. USA* 116, 15849–15854 (2019).

[2] J. Bergstra and Y. Bengio, Random search for hyper-parameter optimization, *J. Mach. Learn. Res.* 13, 281–305 (2012).

[3] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer (2009).

[4] J. Hoffmann *et al.*, Training compute-optimal large language models, *NeurIPS* (2022).

[5] J. Kaplan *et al.*, Scaling laws for neural language models, arXiv:2001.08361 (2020).

[6] J. R. Klein and A. Roodman, Blind analysis in nuclear and particle physics, *Annu. Rev. Nucl. Part. Sci.* 55, 141–163 (2005).

[7] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press (2022).

[8] P. Nakkiran *et al.*, Deep double descent: where bigger models and more data hurt, *ICLR* (2020).

[9] L. Ouyang *et al.*, Training language models to follow instructions with human feedback, *NeurIPS* (2022).

[10] B. Recht, R. Roelofs, L. Schmidt and V. Shankar, Do ImageNet classifiers generalize to ImageNet?, *ICML* (2019).

[11] C. Zhang, S. Bengio, M. Hardt, B. Recht and O. Vinyals, Understanding deep learning requires rethinking generalization, *ICLR* (2017).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.